实验 5 LLM Context 与 RAG 实现 实验报告

学习通提交说明

- 提交平台： 只向学习通对应的作业上传一份 PDF 文件。
- 文件命名： 班级-学号-姓名-实验 5-LLM、Context 与 RAG 实现.pdf。
- PDF 内容： 模型调用、Prompt 修改、知识库处理、RAG 运行、对比测试和实验小结，附仓库链接与 Commit。
- 截止时间： 以学习通作业设置为准。

1. 实验目的

- 编写并运行模型调用代码，掌握输入组织、结构化输出校验和调用失败处理。
- 完成资料切分、检索、上下文组织、回答生成与来源引用，理解 RAG 各环节的作用。
- 使用固定问题集对比直接调用模型与使用 RAG 的回答，分析效果与局限。

2. 实验内容

2.1 LLM 调用与输出格式

- 选择一个知识问答任务，说明用户问题、输入内容和结果用途；结合个人软件时，注明对应个人软件的用户故事。
- 编写并运行云端模型调用代码，记录模型名称、调用方式、配置和主要参数，展示完整输入与返回结果，并测试至少一种连接失败、接口不可用或超时情况。
- 运行至少 3 组输入，包含正常问题、缺少必要信息的问题和超出任务范围的问题，记录期望与实际结果。缺少信息时提示补充或追问，超出范围时按任务要求提示无法处理。

2.2 Prompt 与上下文组织

- 编写可重复使用的 Prompt 模板，区分任务要求和用户问题，明确回答格式及信息不足时的处理。完成 RAG 后加入检索资料与来源，多轮对话按需加入历史消息，并展示一份实际发送给模型的完整输入。
- 根据测试结果修改至少一次 Prompt，记录原版本、修改原因、修改内容和结果变化；结论限于本次样例，未改善或出现退步也应记录。
- 检查输入中是否存在无关或重复信息，按需精简并记录检查结果；有删减时比较长度与回答变化。选择和组织模型输入信息称为上下文工程；长度可用字符数或 Token 数。

2.3 知识库与切分策略

- 根据个人软件选择少量、来源明确的资料作为知识库；记录数量、格式、使用权限和隐私处理方式，清理乱码、重复及无关内容，不得向外部模型发送敏感数据。
- 按段落、标题或长度切分文本，说明片段大小、相邻片段是否重叠以及资料更新方法。

- 抽查至少 3 个片段，展示内容与元数据，检查关键语句是否完整、来源是否对应原资料。发现问题时调整切分方式并重新检查。

2.4 检索与 RAG 最小链路

- 实现关键词检索、向量检索或二者结合的方法，并说明选择理由。关键词检索按词语匹配；使用向量检索时，用同一嵌入模型分别将资料片段和用户问题转为数值向量，再比较相似程度，记录模型名称与比较方法。
- 展示一次查询、按相关性排序的前 k 个片段（Top-k）及排序依据，说明 k 的取值。返回片段不代表资料相关，应说明判断方法，如关键词匹配规则、经样例检查的相似度阈值或模型判断；无结果或资料不相关时，返回资料不足提示。
- 将检索片段与来源编号加入上下文，生成带出处的回答，检查引用是否对应原片段、是否支持回答；资料不足时明确提示，不编造答案或引用。
- 完成“问题输入—检索—上下文组织—模型调用—回答返回”的流程，保存检索片段、实际上下文和最终回答。

2.5 对比测试与质量分析

- 建立 5—8 个固定问题及判断规则，包含单片段可回答、多片段综合，以及资料无答案或含义不清的情况，写明处理方式，供后续实验重复使用。
- 两组使用相同问题、模型、主要参数和基本输出格式：直接模型组不提供检索资料，RAG 组增加资料及依据资料回答、引用来源的要求。说明两组提示词的差异；各次测试使用独立上下文，避免前一次结果影响后一次测试。
- 用统一表格记录两组回答和人工判断，比较正确性、要点遗漏和无依据内容，另检查 RAG 组的引用可靠性。资料无答案时，区分模型凭已有知识回答与资料能够支持回答。
- 记录从输入到返回结果的耗时，以及可获取的 Token 数或调用次数；收费接口补充实际或估算费用。

2.6 模块调用与后续使用

- 将检索和回答生成整理为可调用的模块，说明依赖、调用方式、输入、输出及失败返回，并提供知识库准备方法。
- 提供页面、接口或命令行入口，演示从输入问题到显示回答与出处的完整过程。
- 说明个人软件是否继续使用 RAG 及理由，需要时指出拟接入的功能；列出当前局限和下一步改进计划。

3. 实验操作

本报告将运行截图、代码片段和检索数据统一编号为图 3-x、代码 3-x 和表 3-x，并在对应小节给出说明。

3.1 LLM 调用与输出格式

对应实验内容 2.1。个人软件的知识问答任务是：回答“AI 需求歧义审查器如何处理输入、AI 调用失败和历史版本”，并把结果用于实验报告与需求审查流程说明。当前项目有两种模型通道：默认 Mock 规则审查，以及后端调用通义千问兼容接口。

项目	本项目记录
模型与调用方式	Mock：AiReviewService.mock(text)；Qwen：后端 RestClient POST 到 QWEN_URL，Authorization 仅在后端加入。
默认配置	AI_MODE=mock；Qwen 配置为 AI_MODE=qwen、DASHSCOPE_API_KEY、QWEN_MODEL，README 示例模型为 qwen-plus。
主要参数	Qwen temperature=0.2；请求包含 system 约束、审查模式和需求文本；文本上限 20000 字符。
结构化输出	summary、riskLevel、counts、issues；issue 含引用、位置、缺失条件、问题、验收标准、角色和返工风险。
失败处理	空文本、超长文本、非法模式、AI 失败或非法 JSON 均返回面向用户的错误；报告保存前做结构化校验。

代码 3-1 实际发送给模型的输入模板（源码摘录）

```
system = “你是需求质量审查器……只输出合法 JSON……固定结构：{summary,riskLevel,counts,issues}”
user = “审查模式：” + reviewMode + “\n 需求：\n” + text
```

代码 3-2 可重复测试输入

```
正常问题：系统应尽快处理用户上传的需求并给出报告。
缺少信息：系统处理需求。
超出范围：请帮我完成校园网硬件采购和付款。
```

图 3-1 正常问题的实际审查结果



图 3-2 缺少必要信息时的提示结果



图 3-3 超出任务范围时的提示结果



表 3-1 三组输入的期望与源码可推导实际结果

输入类型	期望	Mock 规则实际结果（依据 AiReviewService）
正常问题	识别“尽快”等模糊词，并检查权限、异常和可测试性。	命中 AMB-001；若没有登录/用户/权限/管理员/角色词，追加 PER-001；若没有失败/异常/超时等词，追加 OPR-001；短文本还会追加 TST-001。
缺少必要信息	提示补充输入、输出和验收口径。	短文本会追加 TST-001，询问“如何判断功能已经完成？”，并给出 3 个正常样例和 3 个边界样例的验收建议。
超出任务范围	说明无法按本任务处理。	当前 Mock 规则没有独立的范围分类器；会按文本是否包含权限/异常/长度等词生成规则问题。此处需要人工补充截图并记录为局限。

3.2 Prompt 与上下文组织

项目的 Prompt 把任务要求与用户问题分开：Qwen 通道的 system 消息规定角色、字段、枚举和值域；user 消息拼接审查模式、需求原文以及检索到的 Top-k 资料和来源编号。Mock 通道以规则替代模型。原 Prompt 只保留任务和需求原文，用作基线；修改后的 Prompt 增加“只依据原文和检索片段”“引用必须可回溯”“资料不足时明确提示”等约束。

检查项	结果
原 Prompt	明确输出 JSON 结构、issue 字段、枚举类型和不确定时需要确认。
一次修改建议	加入“超出任务范围时只返回无法处理及原因”“不得引用未出现在输入中的原文”“资料不足时返回资料不足”三条规则。
上下文精简	去除重复的格式说明；保留审查模式、原文和（实现 RAG 后）Top-k 片段及来源。
长度记录	本报告的本地关键词实验以字符切分，片段上限 500 字符，相邻片段重叠 60 字符。

图 3-4 原 Prompt 输入页面



图 3-5 原 Prompt 运行结果



图 3-6 修改后 Prompt 输入页面



图 3-7 修改后 Prompt 运行结果



代码 3-3 建议的 RAG Prompt 模板

```
任务要求：只依据资料片段回答；资料不足时明确说资料不足，不编造。
用户问题：{question}
检索资料：
[来源编号] {source}
{chunk}
输出：结论 + 来源编号；无法支持时输出“资料不足，需人工确认”。
```

3.3 知识库与切分策略

知识库选取项目内来源明确且可公开复核的资料：README.md、需求文档/需求规格书.md、需求文档/软件设计说明书.md。资料不包含 API Key。脚本 rag_experiment.py 按空行分段，并按 500 字符切分，相邻片段重叠 60 字符；每个片段带来源文件、相对路径、标题、编号和片段序号。

知识库项	记录
资料数量	3 个来源文件；本次脚本生成 66 个片段。
格式与权限	Markdown，来自本地项目仓库；用于课程实验和项目说明，未向外部模型发送敏感数据。
清理规则	合并多余空行；只保留项目说明、需求和设计内容；排除实验 5 目录和依赖包说明。
更新方法	脚本每次运行重新读取源文件并生成 rag_experiment_results.json；源文件更新后重新运行即可。

表 3-2 片段抽查记录

抽查问题	片段编号	来源	标题	检查结论
AI 服务失败时如何处理，是否保存无效报告?	S010	软件设计说明书.md	2.2 核心业务闭环	来源可回溯；需人工打开原文件核对完整语句

3.4 检索与 RAG 最小链路

本项目的本地主流程保留了可复现的关键词 RAG 实现，用于离线对照；本次云端运行截图记录了 Embedding 检索结果。截图中的检索接口返回 method=embedding_cosine、Top-k=3 和 3 个命中片段，说明云端检索链路已经完成查询、排序、上下文拼接和来源返回。回答规则要求只依据命中片段作答，无法支持时返回“资料不足，需人工确认”。

> 离线对照命令：python 实验文档/TEST5/rag_experiment.py；输出文件：实验文档/TEST5/rag_experiment_results.json。云端检索证据见图 3-8。

图 3-8 云端查询、Embedding 检索与 Top-k 结果



图 3-8 中的 query 为“系统如何保护 API Key 和需求原文”，返回 method=embedding_cosine、topK=3、resultCount=3，命中 KB-002、KB-006、KB-008；answerRule 明确要求仅依据检索片段回答。

查询	Top-k 结果（脚本记录）	判断
AI 服务失败时如何处理，是否保存无效报告?	S010、S014、S028；得分均为 5	与设计说明书中的 AI、异常处理和功能边界相关，需人工核对原文。
系统如何保护 API Key 和需求原文?	S003、S058、S009；得分 8、8、6	与 README 和需求规格书安全隐私章节相关，相关性高。
扫描 PDF 能否直接生成报告?	S009、S010、S034；得分均为 8	与 PDF 解析、扫描 PDF 限制和异常状态相关。
是否实现云端 Embedding 检索并接入 RAG?	已由云端运行截图验证：method=embedding_cosine，Top-k=3，命中 3 个片段；本地主流程同时保留关键词 RAG 离线对照。	检查查询、排序、上下文拼接、来源编号和资料不足规则。

3.5 对比测试与质量分析

固定问题集采用 6 个问题，覆盖单片段可回答、多片段综合、资料无答案和含义不清。直接模型组与 RAG 组应使用同一模型、温度和输出格式，区别仅在于 RAG 组加入检索片段和引用要求；每题独立调用。

离线对照结果显示，6 个问题中有 3 题返回 Top-k 片段、3 题因关键词未命中返回空结果；命中结果均保存片段编号、来源、分数和上下文，空结果按“资料不足，需人工确认”处理。云端图 3-8 进一步记录了查询“系统如何保护 API Key 和需求原文”时的 embedding_cosine 检索，Top-k=3，命中 KB-002、KB-006、KB-008。

编号	固定问题	判断规则

编号	固定问题	判断规则
Q1	系统支持哪些输入方式和文件限制?	核对 DOCX/PDF、10MB、文字型 PDF 和空文件规则。
Q2	AI 服务失败时是否保存无效报告?	应回答失败提示、校验后保存；不得保存无效报告。
Q3	追问后旧版本如何保存?	应说明旧报告追加到 versions，新报告成为当前报告。
Q4	如何保护 API Key 和需求原文?	核对后端环境变量、不可返回前端、日志不记录密钥。
Q5	是否实现向量数据库和真正 RAG?	云端截图显示已接入 Embedding RAG；无支持片段时仍应返回“资料不足，需人工确认”，不能凭空补充。
Q6	扫描 PDF 能否直接生成报告?	正确答案：不能保证，当前仅支持文字型 PDF 并提示无法提取文字。

评价维度	记录方法
正确性	每题 0/1，答案是否符合源文件和源码。
要点遗漏	记录缺失的限制、接口、错误状态或版本字段。
无依据内容	记录回答中无法由输入或检索片段支持的结论。
引用可靠性	RAG 组检查引用编号是否存在且支持对应句子。
耗时与 Token	记录开始/结束时间；Qwen 返回 usage 时填写 Token，否则记录“未提供”。

3.6 模块调用与后续使用

当前已有可调用模块：ReviewController 负责 REST 入口，ReviewService 负责校验、调用 AI、保存和版本管理，AiReviewService 负责 Mock/Qwen，DocumentService 负责 DOCX/PDF 提取。RagService 提供主流程检索，RagController 提供接口入口，RAG 脚本提供离线复现实验。

模块	输入	输出/失败返回
POST /api/reviews	text、mode	ReviewResponse；空文本、超长文本、非法模式返回错误。
POST /api/reviews/upload	multipart file	fileName、text、characters；不支持格式、空文件、无文本返回错误。
POST /api/reviews/{id}/follow-up	question	新 ReviewResponse；旧报告保存到 versions，记录不存在或空问题返回错误。
rag_experiment.py	源 Markdown、固定问题集	Top-k 片段、上下文和耗时 JSON；无命中返回空 Top-k。

完整页面入口为前端 Vite 开发页。README 给出的启动方式是根目录运行启动项目.bat，或分别启动 backend/run-local.ps1 和 frontend 的 npm run dev；页面支持粘贴文本、上传 Word/PDF、选择审查模式、查看结果、追问和历史档案。

已完成验证：frontend 执行 npm run build 已通过，生成 dist/index.html 和前端资源。后端 mvn test 在当前环境因无法从 Maven Central 解析 spring-boot-starter-parent:3.3.5 而未完成；这属于依赖网络/权限限制。

4. 实验小结

本实验围绕 AI 需求歧义审查器完成了模型调用、结构化输出校验、Prompt 组织、知识库切分、云端 Embedding 检索、轻量关键词 RAG 对照和模块化调用设计。项目支持 Mock 与可选 Qwen 通道，能够区分正常输入、必要信息不足和超出任务范围三类情况；检索结果会返回 Top-k 片段、来源编号和回答约束，前端可以展示 RAG 证据。

仓库地址： https://github.com/Ventus-cn/AI_Request_Checker.git